



CMSC 471 — Introduction to Artificial Intelligence

Programming Assignment 1: A* Search and Heuristics with DOGCAT

Instructor: Dr. KMA Soliman
Total: 100 points
Date: See the course site

Learning objectives

After completing the assignment, you should be able to:

- formulate a search problem using the AIMA Problem interface;
- implement state transitions, goal testing, and evaluation path costs;
- design and justify admissible heuristics for different cost functions;
- compare uniform-cost search and A* empirically;
- distinguish action irreversibility from cost symmetry; and
- critically verify an AI-generated heuristic.

1. The DOGCAT game

DOGCAT is a word-transformation game. You are given two words with the same number of letters. Your goal is to transform the initial word into the goal word by replacing one letter at a time.

Every intermediate result must be a legal word. For this assignment, a word is considered legal if it appears in the provided `scrabble.txt` file dictionary.

For example:

dog → ong → oot → oat

This is legal because each action changes exactly one letter and every resulting word is in the provided dictionary.

The following is not a legal one-step transformation:

oat → tac

because two letter positions change.

For this assignment:

- status are lowercase three- or four-letter words;
- the initial and goal words must have the same length;
- both must appear in the provided dictionary; and
- every action preserves the length of the word.

2. Cost measures

Your implementation must support three ways of measuring path cost.

2.1 Steps

Every legal letter replacement costs 1. Therefore, the total cost is the number of actions in the solution.

For example:

dog → ong → oot → oat

has a steps cost of 3.

2.2 Scrabble cost

The cost of an action is determined by the value of the replacement letter:

Replacement letter	Cost
A, E, I, O, U, L, N, S, T, R	1
D, G	2
B, C, M, P	3
F, H, V, W, Y	4
K	5
J, X	6
Q, Z	10

For example:

dog → ong → oot → oat

has a Scrabble cost of:

$$3 + 1 + 1 + 5$$

because the newly inserted letters are e, t, and a.

Use the values specified above, even if another Scrabble reference uses different values.

2.3 Frequency cost

The dictionary associates each word with a nonnegative rarity value.

When an action produces word w , its cost is:

$$1 + \text{rarity}(w)$$

The initial word does not contribute a frequency cost. Each resulting word does.

3. Starter code and environment

Your repository contains:

- `do.py`, the file you must complete;
- `doctest.py`, which solves and displays individual problems;
- `dict.txt`, which contains the provided dictionary;
- `scrabble.txt`, the provided dictionary and rarity data;
- `search.py` and `utila.py`, the supplied AIMA search files; and
- any additional supporting files included in the repository.

Use Python 3.

Use the supplied versions of `search.py`, `utila.py`, and the other starter files. Keep them in the same directory as `do.py`. Do not replace them with files downloaded from another version of the AIMA repository.

4. Programming requirements [60 points]

Complete the `do` class in `do.py`. It must extend the supplied AIMA Problem class and solve any valid three- or four-letter DOGCAT instance.

Complete the following methods:

...`init`...

Initialize the problem using:

- the initial word;
- the goal word; and

2

- one of the cost measures: steps, scrabble, or frequency.

Validate the supplied arguments. The initial and goal words must:

- be lowercase strings;
- contain either three or four letters;
- have the same length; and
- appear in the provided dictionary.

The cost argument must be one of the three supported values.

actions

Returns all legal one-letter replacement actions from the given state.

A legal action must:

- change exactly one position;
- replace the existing character with a different character; and
- produce a word contained in the provided dictionary.

Use the action representation specified in the starter code.

result

Returns the state produced by applying a legal action to the current state.

goal, test

Returns True exactly when the given state equals the goal state.

path, cost

Returns the evaluation cost of reaching the new state.

Its behavior must depend on the selected cost measure:

- steps: add 1;
- scrabble: add the assigned value of the newly inserted letter; or
- frequency: add $1 + \text{rarity}(w)$, where w is the resulting word.

h

Returns a nontrivial admissible estimate of the remaining cost.

Your heuristic must:

- depend on the selected cost measure;
- return 0 at the goal;
- never overestimate the true minimum remaining cost; and
- not be the trivial heuristic that always returns 0.

You will define and justify each heuristic in the written portion.

...`repr`...

Returns a clear string representation of the problem instance, including the initial word, goal word, and cost measure.

You may write auxiliary functions when useful.

3

6, Q1)
Steps $L = \text{Length of word}$ $S = \text{State}$
 $I = \text{index}$ $G = \text{Goal}$

$$1) h_{\text{steps}}(S) = \sum_{i=0}^L 1(S[I] \neq G[I])$$

2) It lower-bounds the minimum # of character swaps/actions to reach the goal word.

3) With the steps cost metric, every legal action of changing one character in the word with an added path cost of 1. It cannot overestimate the true cost because its estimated cost is the best-case scenario of getting to the goal word, with fixing each mismatched character. Any extra actions add cost making it always $\geq h_{\text{CS}}$.

Scrabble

$$1) h_{\text{scrabble}}(S) = \sum_{\text{mismatches}} \text{Scrabble}(\text{Goal}[I])$$

2) It lower-bounds the minimum total Scrabble costs of the letters required to reach the goal word.

3) To reach the goal, every wrong letter must eventually be replaced by the goal's letter. The best-case scenario is swapping directly to that target letter, which costs its Scrabble value. Any extra steps will only increase the cost, keeping $h_{\text{CS}} \leq \text{actual cost}$.

Frequency

$$1) h_{\text{CS}} = \# \text{ of mismatches} \cdot 1.0$$

2) It lower-bounds the minimum path cost achieved through character swaps/actions to reach the goal word.

3) The true cost cannot be overestimated because

the # of mismatches is always going to be less than the frequency cost, which is $1 + \text{rarity}$.
With 1 mismatch, $1 < (1 + \text{rarity})$.

6, Q2)

1) YES, UCS and A* always gave the same optimal cost.

2) Why → ASK (Scrabble)
Bear → DUCK (Scrabble)

3) Why → ASK (Frequency)

Bear → DUCK (Frequency)

4) Scrabble heuristic provides a much

3

5. Testing your implementation

Test your code using `test-answers.py` and `testcost.py`. Example commands include:

```
python testanswers.py dog on
python testcost.py why ask steps
python testcost.py bear duck frequency
python testcost.py
```

python testcost.py dog on
d(dogcoststep) cost: 3.00; time:0.000; solution: dog cost on; deltas[0,0,0,0]

python testcost.py why ask
d(whyaskstep) cost: 3.00; time:0.000; solution: why who also also ask; deltas[2, -3, -3,0,0,0]

python testcost.py test back
d(testbackstep) cost: 3.00; time:0.000; solution: test back test back; deltas[0,0,0,0,0]

python testcost.py how how
d(howhowstep) cost: 3.00; time:0.000; solution: how how how how; deltas[0,0,0,0,0]

python testcost.py bear duck steps
d(bearduckstep) cost: 0.00; time:0.000; solution: bear back back back; deltas[0,0,0,0,0,0]

python testcost.py bear duck scramble
d(bearduckscramble) cost: 11.00; time:0.000; solution: bear back back back; deltas[0,0,0,0,0,0]

python testcost.py bear duck frequency
d(bearduckfrequency) cost: 30.26; time:1.035; solution: bear back back back; deltas[27.011, -15.546, -7.094,0,0]

Your cost routines may differ. Your program may also return a different optimal path, and your deltas may differ depending on the admissible heuristic you implement.

- Under the steps objective, an optimal alternative must have the same number of steps.
- Under the Scramble and frequency objectives, an optimal alternative must have the same total cost, but it may contain a different number of steps.

Interpreting deltas

A positive delta proves that the heuristic overestimated at that node. Zero or negative deltas are expected for an admissible heuristic, but observing them only along one returned path does not prove that the heuristic is admissible for every state.

Measuring nodes expanded

For the experiments in Written Question 2, use the reporting behavior already available in the supplied search functions:

```
uniform_cost_search(problem, display=True)
astar_search(problem, display=True)
```

Run every search using a new UC problem instance.

4

For this assignment, **nodes expanded** means the number reported by the supplied search function when `display=True`. (Do not use `result.n` as the primary measure of heuristic quality because `result.n` can vary across machines and executions.)

6. Written analysis [40 points]

Submit your answers in a PDF named `path6.pdf`.

Support your conclusions using your implementation, formal reasoning, and experimental results. A returned path may differ from the sample output while still being correct. It'll have the same optimal cost.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—**steps**, **Scramble**, and **frequency**—describe the heuristic implemented in your program.

For each heuristic:

- Give a precise formula or pseudocode.
- Explain what **unavoidable** remaining cost it lower-bounds.
- Explain why it **cannot** overestimate the true optimal remaining cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

Question 2: Does the heuristic actually help? [10 points]

Run both **uniform-cost search** and **A*** on:

- WFF + ASB
- WBA + DECK

Run each problem using all three cost measures. Report:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY + ASK	Steps	5	713	5	16	97.83%
WHY + ASK	Scramble	9	765	9	13	98.30%
WHY + ASK	Frequency	39.96	808	36.86	755	6.56%
BEAR + DECK	Steps	4	641	4	6	99.13%
BEAR + DECK	Scramble	11	246	11	6	99.80%
BEAR + DECK	Frequency	36.14	1084	36.24	720	33.50%

Calculate the percentage reduction as:

$$100 \times \frac{\text{UCS expansions} - \text{A* expansions}}{\text{UCS expansions}}$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

- Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.
- For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

5

3. For each word pair, under which cost measure was the heuristic least informative?

4. Explain the differences using the definition of your heuristic, not machine alone.

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGOAT actions are reversible. If one legal word can be changed into another, the reverse change is also legal.

For each cost measure—**steps**, **Scramble**, and **frequency**—determine whether the optimal cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

- State **yes** or **no**.
- Justify your conclusion.
- If the answer is **no**, give a concrete counterexample using words from the provided dictionary.

Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Question 4: LLM heuristic costs [10 points]

Give a generative AI system the following prompt:

Is DOGOAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Enforce the frequency objective, rating word w as $\text{cost}(w) \propto \text{rarity}(w)$, where the multiplicative rarity value is defined in the dictionary.

The baseline heuristic $h(w)$ is the number of positions in which state w differs from the goal. Propose a **redundant** heuristic intended to be more informative than $h(w)$ while remaining admissible. Define it precisely and justify your proposal.

If “more informative,” we mean that the proposed heuristic should be at least as large as $h(w)$ for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

- The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.
- A precise formula or pseudocode for the proposed heuristic.
- Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.
- If the proposal is invalid, make the smallest repair you can or replace it with a side objective.
- Test the modified or repaired heuristic against A_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes; then conclude whether the evidence supports using the proposed heuristic.

Experimental tests can reveal counterexamples, but proving a collection of tests does not by itself prove admissibility. The AI system is not expected to be correct. Your grade is based on the quality of your verification, correction, and justification.

7. Collaboration, sources, and generative AI policy

You may discuss the assignment’s concepts and possible approaches with classmates. You remain responsible for your final implementation, analysis, and understanding.

You may use:

- course materials and supplied code;
- Python and library documentation;
- other external explanatory resources; and
- generative AI systems.

6

Dear → Uncle Linguist,

4) Scrabble heuristic provides a much tighter lower-bound which means the heuristic value is very close to the actual cost. The Frequency heuristic provides a weak lower-bound which produces values that under-estimate the actual cost.

6, Q3)

Steps

1) Yes

2) The # of actions needed to change a $W_1 \rightarrow W_2$ is the same as the # of mismatches. Going from $W_2 \rightarrow W_1$ has a Symmetrical cost because the # of mismatches is the same.

Scrabble

1) No

2) Based on Scrabble cost, going from

$W_1 \rightarrow W_2$ could replace a lower cost letter with a higher cost letter which is expensive.

In reverse, $W_2 \rightarrow W_1$ would be replacing a higher cost letter with lower cost, which is cheaper, so the cost is asymmetric.

3) $\text{dog} \rightarrow \text{dog}$ Scrabble cost (d) = 2 $\rightarrow 2 < 3$
 $\text{dog} \rightarrow \text{cog}$ Scrabble cost (c) = 3

Frequency

1) No

2) Cost is $1 + \text{rarity}(W)$. Going $W_1 \rightarrow W_2$

where $\text{rarity}(W_2) > \text{rarity}(W_1)$ costs more while going in the reverse $W_2 \rightarrow W_1$, $\text{rarity}(W_2) < \text{rarity}(W_1)$, making the cost asymmetric.

3) $\text{dog} \rightarrow \text{cog}$, $\text{rarity}(\text{cog}) = 11.393166$
 $\text{cog} \rightarrow \text{dog}$, $\text{rarity}(\text{dog}) = 5.819885$

Question 4 requires the use of a generative AI system.

If you use generative AI elsewhere in the assignment, briefly disclose:

1. the system used;
2. what you used it for; and
3. how you verified the resulting code or reasoning.

Complete prompt histories are not required except for the exact prompt and relevant output requested in Question 4.

Clearly cite any external code, text, or substantial solution idea that you copy or adapt. Do not submit another person's work or unattributed external material as your own. Monitor if you have discussed with anyone. You are responsible for the correctness of everything you submit and should be able to explain how your implementation and written answers work.

Student UMBC academic-integrity requirements continue to apply.

8. Submission

Submit: 1. `dc.py`, containing your completed DC implementation; and
2. `pdf0.pdf`, containing your answers to the written-analysis questions.

Ensure that:

- your program runs with the supplied repository files;
- your name and section appear in `pdf0.pdf`;
- all required experiments and tables are complete;
- any external resources are cited; and
- any generative AI use is disclosed as described above.

We may evaluate `dc.py` on additional valid DOGCAT problems not included in the visible tests.

Point summary

Component	Points
DOGCAT implementation	60
Question 1: Heuristics and admissibility	15
Question 2: UCS and A* comparison	10
Question 3: Heuristics and costs	5
Question 4: LLM heuristic results	10
Total	100

$$\text{dog} \rightarrow \text{dog} \mid \text{rarity}(\text{dog}) = 5.819885$$

Model and prompt

I used ChatGPT (GPT-5.6 Luna) and used the exact prompt provided in the assignment.

Proposed heuristic

Let $d(n)$ be the number of positions where n differs from the goal, and let $r_{\min}$ be the smallest rarity value in the dictionary.

For n not equal to the goal:

$$h_R(n) = (d(n) - 1) * (1 + r_{\min}) + (1 + \text{rarity}(\text{goal}))$$

For the goal: $h_R(\text{goal}) = 0$

Admissibility

The heuristic is admissible because $d(n)$ actions are required at minimum, since each action changes only one letter.

The final action must enter the goal, costing $1 + \text{rarity}(\text{goal})$, while every other action costs at least $1 + r_{\min}$.

Therefore, the heuristic is a lower bound on the true remaining cost.

Also, because rarity values are nonnegative, $h_R(n)$ is at least $h_0(n)$. It is strictly larger when $r_{\min} > 0$ or the goal has positive rarity.

Testing

For WHY $\rightarrow$ ASK, both heuristics found the same optimal cost of 38.855. The rarity-aware heuristic expanded 755 nodes compared to 808 with (h_0), which is a reduction of 6.56%.

For BEAR $\rightarrow$ DUCK, both heuristics also found the same optimal cost of 38.240. The rarity-aware heuristic expanded 720 nodes compared to 1084 with (h_0), which is a reduction of 33.58%.

Overall, the rarity-aware heuristic expanded fewer nodes on both problems while still finding the same optimal solution costs. The improvement was larger for BEAR $\rightarrow$ DUCK because the rarity information provided a more useful lower bound for that problem.